围猎银狐 -- 终端安全能力建设共享第1期｜基于 YARA 的特征检测与银狐追踪

腾讯安全威胁情报中心 2025年08月29日 16:31 北京

01 引言

在过去的1个多月里，腾讯安全威胁情报中心（以下简称“我们”）持续追踪并披露了“银狐”的最新动向。从 银狐情报共享第1期｜Att&CK视角下的最新活跃技战术分享到 银狐情报共享第2期｜银狐武器库更新，加装RootKit实现多重隐身，再到 银狐情报共享第3期｜银狐软硬兼施，硬刚百款安全软件外，悄悄藏身杀软信任区，这一系列深度分析帮助大量企业及时发现并处置了银狐威胁，也引发了业界对于银狐攻击检测能力的关注。

其中部分文章对终端安全软件的内存层检测技术进行了讨论[1]，这也从侧面印证了一个事实：**文件层免杀对抗愈加激烈的背景下，内存层检测已经成为终端安全软件当前重点建设的检测引擎。**

在我们分析的大量银狐样本中有超过80%使用了内存加载技术。**攻击者先通过加密混淆绕过文件扫描，然后将真正的恶意代码直接加载到内存并解除伪装、利用合法进程的内存空间或者篡改内存中关键数据结构来实现在内存中执行恶意代码，并规避行为层面防御**，进而实现其攻击目的。这也是传统杀软难以及时发现这类新型攻击的主要原因。

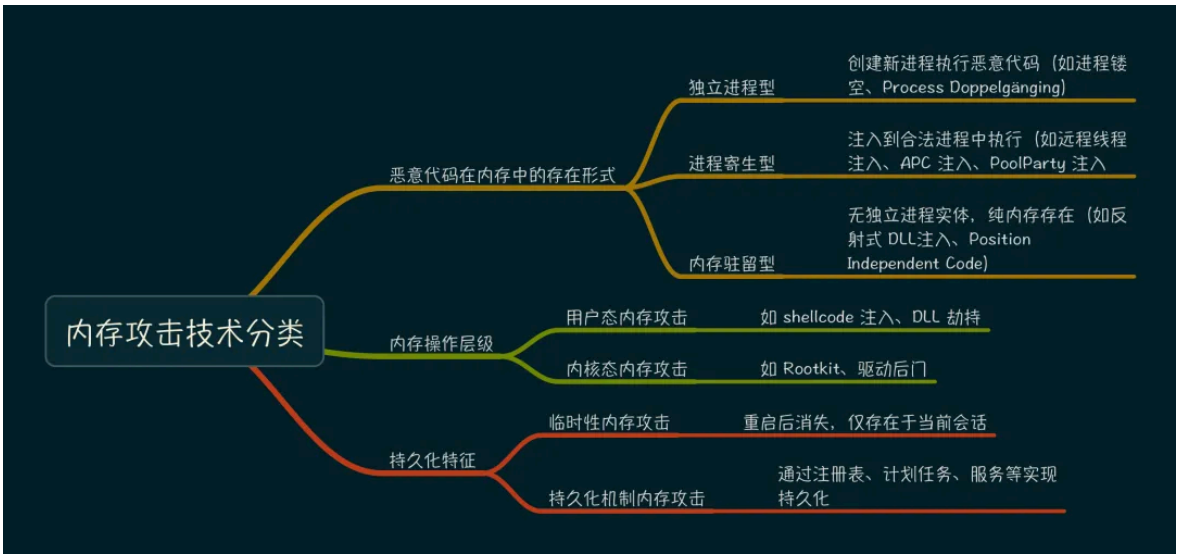
面对这一挑战，腾讯iOA/电脑管家两款终端安全软件的内存检测引擎在长期对抗中提炼出了三个关键认知：

1. **文件免杀 ≠ 内存隐身：**在高阶攻击主要以钓鱼方式获取初始化入侵据点的背景下，文件层免杀已成钓鱼样本的必备操作，也意味着不具备内存层检测引擎的终端安全软件，将无法有效防御钓鱼攻击行为；
2. **单点绕过 ≠ 全面逃逸：**常规内存检测引擎仅设置单点的静态检测特征，攻击者可以通过类似文件免杀的方法测试发现特征点以进行绕过，不具备多点内存特征检测逻辑的终端安全软件，将无法有效应对钓鱼攻防对抗；
3. **引擎强大 ≠ 一劳永逸：**银狐等样本不断进化，终端检测能力也必须保障及时动态更新，不具备云管端等场景下采集数据、筛选分类、深度研判、赋能产品的实时安全情报分析体系支撑的终端安全软件，将无法有效适应钓鱼攻防节奏；

本文将探讨基于用户态的内存威胁检测技术，并以追踪“银狐”为例，重点介绍 iOA 内存检测引擎如何使用 YARA 在内存中精准狩猎，通过实战案例展示内存检测的实际价值。

银狐等现代恶意软件最常用的内存攻击手法包括进程注入、无文件内存马植入、Process Doppelgänging、反射式DLL注入等。这些的攻击手法虽然采用不同的方式为恶意代码注入提供隐蔽性，但是它们的本质都殊途同归：都是为了在内存层面隐蔽地执行恶意代码。正是这些技术的不断迭代，造就了当前异常复杂的内存攻防格局。为了更好地理解和应对这些种类繁多的内存攻击技术，可以根据其实现方式和特征进行多维度分类。

图中将内存攻击技术概括为3个便于大家理解的维度：



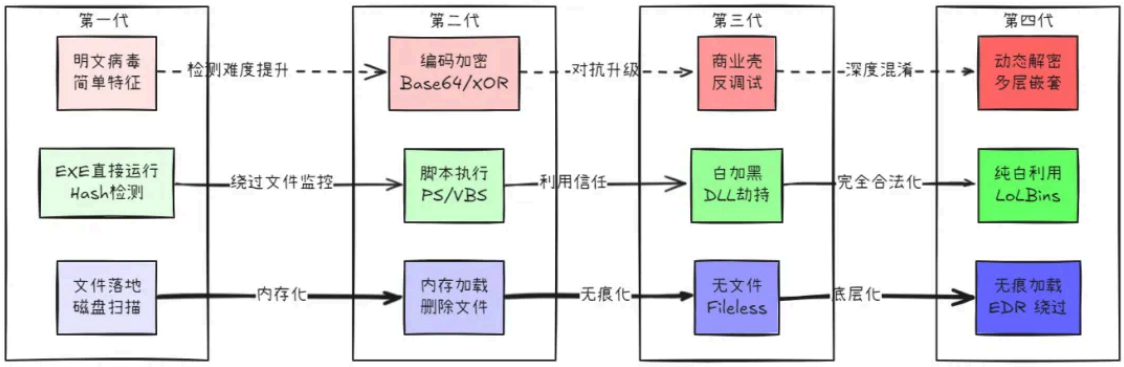
从图中可以看出，现代内存攻击技术呈现出明显的多样化态势，攻击者会根据目标环境和防御措施，灵活组合使用不同维度的内存攻击技术，形成复杂的攻击链。这种多维度的组合技术使得内存威胁的检测和防御变得极具挑战。

同时，由于在Windows平台上，操作系统安全防护机制的不断强化（如PatchGuard、KPP、HVCI等），在带来更多的可靠性的同时也导致了依赖内核模式的安全产品可操作空间大幅缩减。

02 Windows内存检测技术的基础

2.1 Windows 平台威胁技术演变

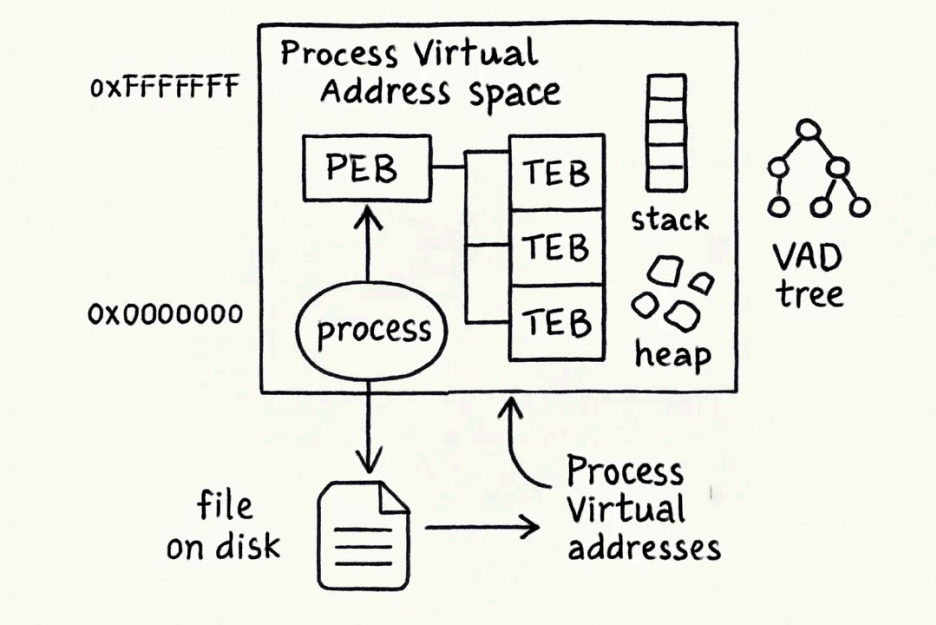
在过去的三十年间，Windows平台上的安全攻防对抗推动了恶意软件技术的快速迭代。如下图所示，威胁技术沿着三条主线并行演进：



威胁技术与检测技术在相互对抗中不断演进。当前阶段，动态解密对抗静态分析、LoLBins实现完全合法化、无痕加载规避EDR已成为主流威胁技术。这些技术的组合运用及其对应的检测方法，构成了当前内存攻防的主战场。

2.2 Windows 内存关键区域

无论威胁技术如何演进，其最终都需要在内存特定区域中执行，这些区域的结构和特性直接影响着检测算法的设计与实现。理解Windows内存管理机制中的重要概念是进行有效内存检测的基础。本节聚焦于与威胁检测最相关的几个关键内存概念，重点说明它们在检测工作中的意义。



文件到进程：从静态到动态的转变

在 Windows 中，当用户执行一个程序时，系统会通过一系列步骤将磁盘上的静态文件转化为内存中的动态进程。这个看似简单的加载过程，实际上是攻防对抗的第一个战场。

正常的加载流程中，PE文件会按照其结构映射到内存，形成进程映像。然而，内存中的映像与磁盘文件往往并不完全一致，一方面它是系统正常运行的需要（如重定位、导入表修复），另一方面也为恶意代码提供了藏身之所。比如：进程镂空会将合法进程掏空然后将恶意代码置入这个进程“外壳”中；反射式DLL会跳过正常的DLL加载流程，直接在内存中“组装”自己。

进程虚拟地址空间：隔离与共享的平衡

每个Windows进程都拥有独立的虚拟地址空间，这个空间是进程所有活动的容器，同样也是了解进程内容的窗口。

PEB和TEB：进程和线程的控制中枢

PEB（Process Environment Block，进程环境块）是Windows为每个进程维护的核心控制结构。包含了进程映像信息、加载模块列表、进程堆的管理信息、进程的各种参数等信息；

TEB（ThreadEnvironmentBlock，线程环境块）则是Windows为每个线程维护的核心控制结构，每个线程都有独立的TEB，而同一进程的所有线程共享同一个PEB。包含了线程的栈信息、线程独有的数据存储TLS（线程本地存储）、PEB指针等各种信息；

内存堆栈：动态内存的两种形态

每个线程都会通过TEB管理自己的栈，栈用于函数调用、局部变量、函数返回地址记录；每个进程都会通过PEB管理进程级的堆，动态内存分配一般都发生在进程的堆中。

VAD树：内存布局的账本

Windows会使用一颗AVL树结构的VAD（虚拟地址描述符）来管理进程的所有内存区域，其节点信息包括了内存区域的地址范围、类型、保护属性、是否关联文件。

内存检测关键点

基于以上内存结构的理解，我们可以得出一个重要结论：即使是最隐蔽的恶意代码，也无法完全脱离Windows内存管理框架而存在。

它们必须申请内存、必须有执行权限、必须存在于某个进程空间中——这些约束条件也是我们检测的切入点。

典型的内存异常特征包括：

- RWX权限的私有内存区域（正常程序极少需要）
- 未在PEB模块列表中的PE映像（可能是反射加载）
- VAD树记录与实际内存不一致（内存隐藏的征兆）
- 跨进程的内存写入操作（典型的注入行为）
- 短时间内大量的内存分配（堆喷射攻击）
- 线程起始地址指向非模块区域（代码注入的标志）

03 基于YARA的内存威胁检测技术

上一章介绍的内存结构为我们提供了检测的理论基础，但在实际检测中仍面临巨大挑战。一个典型的Windows系统有数十上百个进程，并且每个进程的内存空间都不小。恶意代码可能以加密shellcode、反射加载的DLL、无落盘文件的内存PE等多种形态存在。在不影响系统性能的前提下，快速准确地识别这些不同形态的威胁是一个巨大的挑战。

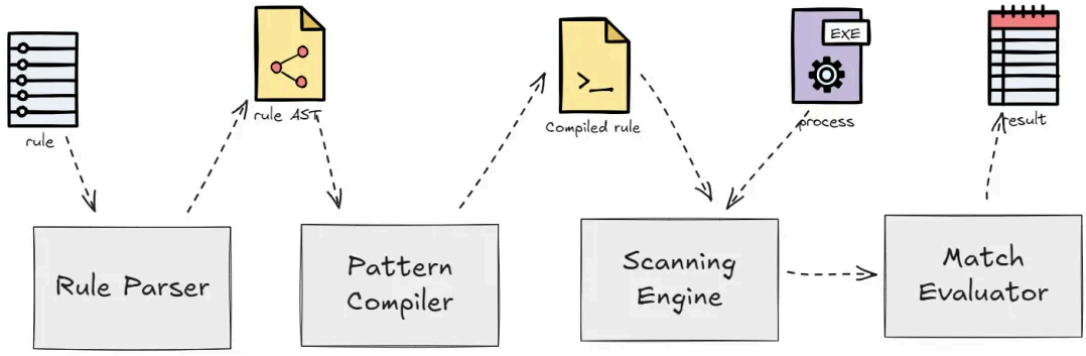
面对这些挑战，YARA[2]作为业界公认的恶意软件识别利器，提供了一套行之有效的解决方案：

- 高性能的模式匹配引擎：采用AC自动机等算法，能在O(n)时间内完成多模式匹配
- 灵活的规则语言：支持字符串、正则、十六进制等多种模式，可精确描述各类特征、
- 丰富的条件逻辑：通过布尔运算、位置约束、计数控制等实现精准判定

3.1 YARA内存扫描的工作原理

YARA从规则到检测的全流程，可以抽象为四大核心组件的协同工作，如下图所示：

- **规则解析器（Rule Parser）**加载规则文本文件，并通过词法分析和语法分析将其转化为AST（抽象语法树）；
- **模式编译器（Pattern Compiler）**会对规则进行优化以提高匹配效率，同时输出编译后的规则文件；
- **扫描引擎（Scanner）**会加载规则并遍历目标进程的内存空间，记录匹配位置和上下文信息；
- **决策评估器（Match Evaluator）**则会根据条件逻辑判断是否触发规则。



其核心扫描引擎在对进程进行内存扫描时，实际上经历了以下步骤:

```
// YARA 内存扫描流程简化版
// 1. 附加进程
HANDLE hProcess =OpenProcess(PROCESS_VM_READ, FALSE, pid);
// 2. 遍历内存
void* addr =0x00400000;// 起始地址
while(addr <0x7FFFFFFF){
  MEMORY_BASIC_INFORMATION mbi;
// 查询内存块信息
VirtualQueryEx(hProcess, addr,&mbi,sizeof(mbi));
// 如果内存可读
if(mbi.State == MEM_COMMIT){
// 读取内存内容到缓冲区
    BYTE* buffer =(BYTE*)malloc(mbi.RegionSize);
if(ReadProcessMemory(hProcess, addr, buffer, mbi.RegionSize,NULL)){
// 使用YARA规则扫描内存
yr_rules_scan_mem(rules, buffer, mbi.RegionSize,
            SCAN_FLAGS_PROCESS_MEMORY,
            scan_callback,NULL,0);
}
free(buffer);
}
// 下一个内存区域
  addr += mbi.RegionSize;
}
CloseHandle(hProcess);
```

3.2 内存检测规则的设计艺术

不同的检测场景对YARA规则有不同要求。攻击检测注重实时性，取证分析注重准确性，威胁狩猎注重覆盖面。这种差异导致规则的设计标准和质量评价体系各不相同。

实践中，盲目混用不同场景的规则往往会带来性能、误报等问题。因此，理解YARA规则的不同使用模式至关重要。

精准特征检测

通过寻找恶意软件留下来的“指纹”可以精准识别恶意软件。

这些“指纹”一般会是硬编码的C2服务器地址、特定的字符串常量、特征的密钥，或者是特定恶意软件中留下来的代码模式。

```
rule SilverFox_Signature {
meta:
    layer = "signature"
    confidence = "high"
strings:
```

```
    $mutex = "Global\\SilverFox_2024_Mutex"
    $marker = { 53 46 58 00 }  // 'SFX\x00'
condition:
    all of them
}
```

精准检测的精准度极高，几乎不会产生误报。当你找到了某个恶意软件家族的独特特征，就像找到了罪犯的指纹，可以非常确定地判定其身份。但这种方法也有明显的局限性：攻击者只需要稍微修改这些特征，比如更换C2地址、使用不同的加密密钥，就能轻易绕过检测。

行为模式检测

行为检测不关注恶意代码“是什么”，而是关注它们“做了什么”。

比如同时申请可读可写可执行（RWX）权限的内存、在短时间内解密大量代码、创建远程线程并注入代码、修改关键系统数据结构等。这些行为单独看可能是正常的，但组合在一起就高度可疑。

```
rule Generic_Injection_Pattern {
meta:
    layer = "behavior"
    confidence = "medium"
strings:
    $inject_chain = /VirtualAllocEx.{0,200}WriteProcessMemory.{0,100}CreateRemoteThread/
condition:
    $inject_chain
}
```

行为规则能够检测未知威胁和变种。即使是全新的恶意软件，只要它采用了类似的攻击技术，就会被行为规则捕获。

但这种方法也容易产生误报，因为某些合法软件也可能表现出类似的行为。因此，行为规则需要更精细的条件设计和上下文判断。

攻击取证

取证往往用于寻找攻击活动留下来的“证据”。不是事中检测预警，而是事后分析发现那些表明“这里发生过什么”的证据。

```
rule Anomaly_Unmapped_PE {
  meta:
    layer ="anomaly"
    requires_context =true
  condition:
    pe.is_pe andnot pe.is_dll()and
notdefined(pe.linker_version)// 可能是手工构造
}
```

取证规则更适合事后分析和威胁狩猎。当你怀疑系统已经被入侵但不知道具体是什么威胁时，取证规则能够帮你找到攻击的蛛丝马迹。

比如，通过扫描所有进程的内存，寻找那些不在PEB模块列表中却有PE结构的内存区域，往往能发现隐藏的恶意DLL。

04 银狐狩猎：内存检测实战

4.1 银狐木马技术剖析

从往期的文章中，我们总结了银狐攻击者惯用的攻击手法：充分利用社会工程学诱导用户执行恶意载荷，通过白利用技术规避安全检测，并在内存层面实施多重隐身策略。

作为一个高度活跃的威胁，银狐在文件层面的伪装手法层出不穷，但其在内存中的行为模式却相对稳定，这一点，恰好这为我们的检测提供了可靠的切入点。

从内存检测视角看银狐的核心技术栈

1. 白利用与DLL劫持:银狐偏好利用带有合法数字签名的程序（如迅雷模块、Microsoft UevAppMonitor.exe等）进行DLL侧加载。这种技术在内存中会呈现特殊模式：合法进程的内存空间中会出现非预期的DLL映射，这些DLL虽然在PEB模块列表中，但其代码段内容与原始文件存在显著差异。
2. 多样化的内存注入技术:银狐的注入技术库极其丰富，包括远程线程注入、APC注入、PoolParty注入等。无论采用何种注入方式，其在内存中的表现具有共性特征，这些特征对应了我们在3.3节讨论的行为模式检测策略：
   a. 目标进程（如explorer.exe、lsass.exe）出现RWX权限的私有内存区域
   b. VAD树中存在未关联文件的可执行内存块
   c. 新创建线程的起始地址指向非模块区域
3. 载荷隐写与动态解密 银狐将恶意载荷隐藏在图片文件（如Consys21.png）或注册表中，运行时在内存中动态提取。这个过程会产生明显的内存活动模式：
   a. 短时间内的大量内存分配操作（解密缓冲区）
   b. 内存属性的动态变化（从RW到RWX）
   c. 解密后出现的PE结构或shellcode特征
4. 深度对抗技术Rootkit模块（如acx01.sys）和"天堂之门"这种内核级攻击手法难以从用户态直接检测，但其与用户态的交互仍会留下痕迹：
   a. 受保护进程的异常终止行为（无法被正常结束）
   b. 32位进程中的64位代码段（天堂之门的标志）
   c. 系统服务表的Hook痕迹（间接证据）
   d. 遗留下的内存特征字符串
5. 持久化机制的内存表现 银狐采用的"文件关联 + 虚拟设备映射 + PendingFileRenameOperations"组合持久化机制虽然巧妙，但在内存中仍有迹可循。

当恶意代码通过这种方式启动时，会在smss.exe或早期系统进程的内存空间中短暂出现异常代码段。

内存检测的关键突破口

基于以上分析，可以得到银狐在内存层面仍存在的三个核心检测窗口：

检测时机	内存特征	yara规则提取点
加载阶段	白利用进程中的异常DLL、解密缓冲区的RWX内存	检测解密算法特征（如sub 0x24、xor 0x25组合）
注入阶段	目标进程的私有可执行内存、异常线程起点	注入API序列和内存属性变化
驻留阶段	未在PEB登记的PE映像、特定互斥体和C2通信字符串	银狐C2套件特有的字符串、配置信息和通信协议

4.2 从特征提取到产品化内存检测

基于上述的技术分析，我们在实际样本分析中提炼出了银狐的多个关键内存特征。这些特征分布在攻击链的不同阶段，为内存检测提供了可靠的锚点。

银狐内存特征提取

如下图所示，银狐样本经常会使用固定的解密算法特征。通过对多个样本分析，可以得到该解密算法内存特征。

```
int __cdecl sub_4064D0(const char *a1)
{
  int i_1; // esi
  int result; // eax
  int i; // ecx
  int v4; // [esp+0h] [ebp-4h] BYREF

  v4 = 0;
  i_1 = sub_4063B0(a1: a1, a2: (void **)&v4);
  result = v4;
  for ( i = 0; i < i_1; ++i )
    *(_BYTE *)(i + result) = (*(_BYTE *)(i + result) - 0x24) ^ 0x25;
  return result;
}
```

同样的，部分银狐样本shellcode特有的配置信息也可以作为内存检测特征。

```
.data:1003EE10                                          ; sub_10011DC0+1A↑o ...
.data:1003EE10                 text "UTF-16LE", '|0:db|0:lk|0:hs|1:ld|0:ll|0:hb|1:pj|:zb|0.1:bb|SPW:'
.data:1003EE76                 text "UTF-16LE", 'zf|1:lc|1:dd|1:3t|:3o|:3p|1:2t|:2o|:2p|1:1t|4898:1o'
.data:1003EEDC                 text "UTF-16LE", '|232.76.712.8:1p|',0
.data:1003EF00                 db    0
```

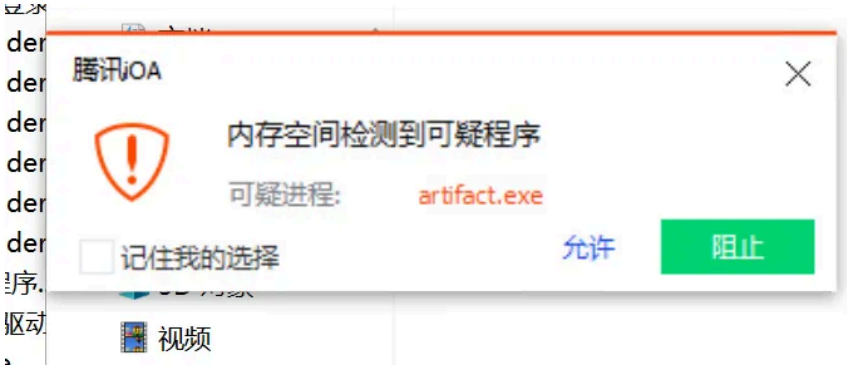
银狐样本内存解密之后通过反射式DLL加载样本的行为也可以作为内存检测特征点。

```
// Token: 0x0600000E RID: 14 RVA: 0x00002260 File Offset: 0x00000460
public static byte[] XorDecrypt(byte[] shellcode, string key)
{
    for (int i = 0; i < shellcode.Length; i++)
    {
        shellcode[i] = (byte)((char)shellcode[i] ^ key[i % key.Length]);
    }
    return shellcode;
}
```

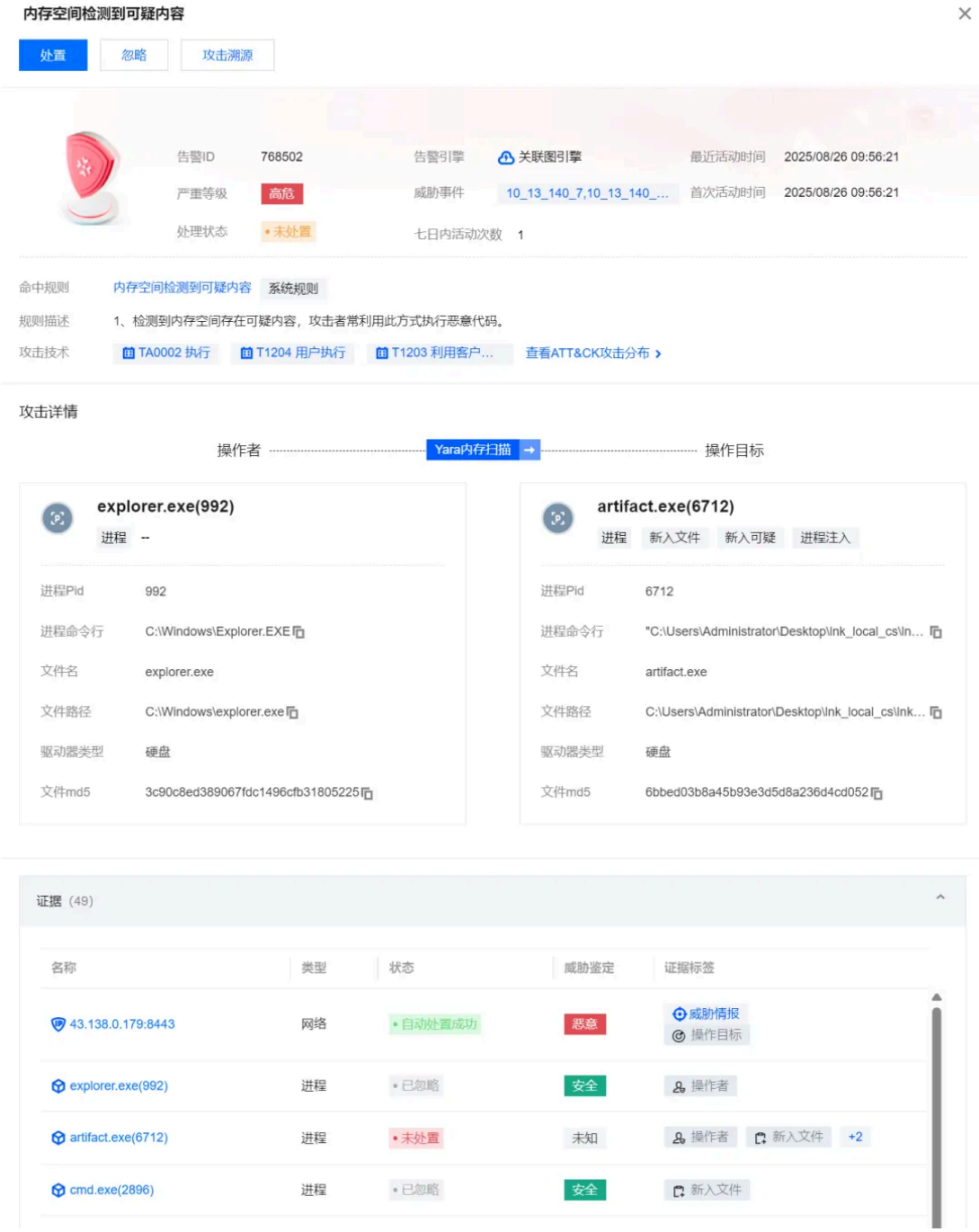
```
// Token: 0x0600000F RID: 15 RVA: 0x00002294 File Offset: 0x00000494
public static void XORedReflectiveDLLInject(int targetId, byte[] buffer)
{
    try
    {
        IntPtr zero = IntPtr.Zero;
        IntPtr zero2 = IntPtr.Zero;
        IntPtr intPtr = ClassExample.OpenProcess(2035711U, false, (uint)targetId);
        IntPtr intPtr2 = ClassExample.VirtualAllocEx(intPtr, IntPtr.Zero, (uint)buffer.Length, 4096U, 64U);
        if (ClassExample.WriteProcessMemory(intPtr, intPtr2, buffer, (uint)buffer.Length, out zero))
        {
            ClassExample.CreateRemoteThread(intPtr, IntPtr.Zero, 0U, intPtr2, IntPtr.Zero, 0U, out zero2);
        }
        else
        {
            Console.WriteLine("[+] Failed to inject shellcode.");
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}
```

实际检测案例展示

腾讯iOA内存检测引擎在监控到内存恶意威胁的第一时间会在客户端产生告警。



同时安全运营人员也能及时在云端发现对应的威胁及时进行研判处理



05 总结与技术展望

本文展示了基于YARA的特征检测在内存威胁狩猎中的能力，但我们也要清醒地认识到，特征匹配只是内存检测的一种手段。

YARA 虽然强大，但其最大的局限在于：特征检测本质上是一种“基于已知抓捕已知”的游戏[3]，我们必须先掌握恶意代码的特征，才能编写规则去将其捕获。面对全新的威胁或高度变形的恶意代码，特别是像银狐这样快速演进的威胁，传统的特征检测往往力不从心。更重要的是，YARA缺乏对内存上下文的理解能力，它无法判断一个内存结构是否合理，也难以识别跨进程的异常行为链。

这就引出了一个关键问题：当攻击者不再使用任何已知特征时，我们该如何检测？

答案指向了内存检测的另一个重要方向：**基于内存结构异常的检测技术**。正如引言中提到的"**单点绕过 ≠ 全面逃逸**"，我们需要构建多维度的检测体系。

在本系列的下篇中，我们将介绍全新的内存检测引擎：QMHunt。与YARA的特征匹配不同，QMHunt通过识别内存中的异常模式来发现内存中的威胁：

检测维度	YARA（特征检测）	QMHunt（异常检测）
检测时机	定时或者触发式扫描	多阶段组合持续监控
检测依据	预定义的恶意特征	偏离正常的内存行为模式
更新频率	需要不断更新规则库	自适应应对新型攻击
绕过难度	修改特征即可绕过	绕过成本极高

YARA 和 QMHunt 并非相互替代，而是优势互补。前者提供高精度的已知威胁检测，后者负责发现未知和变种威胁。两者结合构建起腾讯iOA/电脑管家内存检测引擎的立体防御体系。

在下篇中，我们将通过实际案例详细介绍 QMHunt 的技术原理和检测效果。敬请期待。

06 写在最后

威胁检测是一个不断演进的领域。在内存攻防的战场上，从来没有一劳永逸的“银弹”，只有不断进化的“矛”与“盾”。

本文通过对Windows内存机制和YARA检测原理的深入分析，揭露了一个基本事实：**内存检测本质上仍是特征匹配，只是它能绕过文件层面的加密和混淆，直接在内存中寻找恶意代码的真实面目。**

这并不意味着内存检测就能"无视"所有免杀技巧，攻击者总会找到新的对抗方法，这就是“矛与盾”的永恒博弈。

正因为认识到没有任何单一技术能够一劳永逸地解决所有的问题，我们才持续披露银狐威胁的变种，对抗网络威胁需要的是社区协作，而非产品竞争。

今天大量银狐能够及时地被有效检测，是整个安全社区的共同努力的结果。每一份威胁分析报告、每一个共享的IOC，都在为这场持久战贡献力量。

参考链接

[1] 银狐木马变种难防？天擎“六合”引擎默认内存查杀

[2] GitHub - VirusTotal/yara: The pattern matching swiss knife
https://github.com/VirusTotal/yara

[3] Cobalt Strike and YARA: Can I Have Your Signature? | Cobalt Strike
https://www.cobaltstrike.com/blog/cobalt-strike-and-yara-can-i-have-your-signature